Nominal-free proofs of the theorems Benzmüller's detector flags

Lindheim, Björn

DOI: 10.5281/zenodo.23795772

1 Introduction

Cite as: https://doi.org/10.5281/zenodo.23795772

Setting

Benzmüller and Scott's *Notes* prove variants of Gödel's ontological argument in a shallow embedding of higher-order modal logic in classical higher-order logic [9]. Benzmüller's *Proofs Without Nominals* [6] (v4) points out that the embedding's quantifiers also range over *nominals*, properties like $\lambda u.\, u = w$ that pick out a single world. A proof that substitutes one proves a theorem of the embedding but not necessarily of the modal object language $\mathcal{L}$. v4 ships a detector that flags such "hybrid" proofs in two modes (witness and whole-term), and lists the theorems of its Lean modules it could only prove with a flagged step, among them two of its stated open questions: whether the equivalence of the restricted conjunction axiom Ax1GenTwo with the unrestricted Ax1Gen has a nominal-free proof in variants 2 and 3′, and whether Gödel's lemma L follows under the rigid reading Ax1GenR with no condition on positivity.

Result

For each flagged theorem below we give a new proof of the *identical* statement, from the same module's definitions, that v4's own detector passes in both modes (full names):

- **The equivalence question:** `Ax1Gen_of_Two` and `Two_equivalent`, with `P_of_conj`, `loc_of_conj` and the related lemmas, in `Ax1GenTwoVariant2` and `Ax1GenTwoVariant3Plain` (logic K).
- **The rigid question:** `L_R` in `Ax1GenRigidVariant2`, from Ax2a, Ax2b and Ax4 only, with witness `PosGlobal` and no condition on positivity.
- **Further flagged theorems:** modal collapse and Th5 in `Ax1GenBoxVariant3` (the reflexivity hypothesis unused); fourteen theorems of `GoedelVariantHOML3inS4oneFile`

without `Rrefl` or `Rtrans`; the one-conjunct theorems of `Ax1GenOneVariant3`; collapse and the existence theorems of `Ax1GenTwoVariant3` and `Ax1GenTwoVariant3Th4`; `AllExist` and `MC_OneWorld` of `Ax1GenTwoVariant3Plain`; and collapse in its possibilist and Anderson-quantifier versions.

- **A model:** on two worlds with one always-actual entity and a universal frame, Ax1, Ax2a, Ax2b, Ax3, both forms of Ax4, Ax1Gen with its Two, One and Box restrictions, and Th4 hold, and necessary existence holds at both worlds, so `NoNecExist` fails there; and an abstract Henkin lemma describing what Ax1GenR forces over admitted rigid families.

What is new

v4 already has nominal-free proofs of two of the lemmas listed first (`ae_Two` and `G_ex_Two_unconditional`, under those names), and frame-free `_K` versions of most theorems of `GoedelVariantHOML3inS4oneFile`; what this finding adds there are proofs under the original names that the detector passes, and nominal-free routes to `EssSupp`, `EssDetermined` and `AllExist`. The new results are the equivalence lemmas, which v4 proves only through the nominal arguments `OneWorld` and `R_Id` and lists as open, and `L_R` with no condition on positivity, also listed as open. v4's text and its Lean module disagree on whether the detector flags the witness of its own `L_R` proof; ours passes either way.

Why it matters

v4 asks whether these results are theorems of the modal logic or only of its embedding. A detector-certified proof answers that for every proof the detector reads correctly. The companion findings on this site make the answer semantic: a complete calculus over v4's $\mathcal{L}$-general models, the equivalence and L under the rigid reading proved in every such model, and a hole in the detector that shows why the semantic proofs are needed.

Proved vs checked

Every proof is Lean 4 (core library, axioms propext, Classical.choice and Quot.sound only), built in v4's own modules, and was run through v4's `hybrid_check.py` in witness and whole-term mode with full-name matching. One further claim of the run (a Henkin-model version of the equivalence over eleven listed closure terms) was disputed by the run's skeptic and is left out.

Novelty statement

This result extends:

- C. Benzmüller, "Proofs Without Nominals", arXiv 2609.36279v4, https://arxiv.org/abs/2609.36279: gives detector-certified proofs `ae_Two` and `G_ex_Two_unconditional` of the same statements as `ae_Two_nominal` and `G_ex_Two_unconditional_nominal`,

in both modules. `only_top_and_empty_domain(_False)` follows directly from `ae_Two`, though v4 proves it only through the flagged `OneWorld`. v4 proves Ax1Gen_of_Two, Two_equivalent, P_of_conj and loc_of_conj only through the nominal steps OneWorld/R_Id (flagged), and lists their hybrid-free status as open (sec:plain, sec:open).

- C. Benzmüller, "Proofs Without Nominals", arXiv 2609.36279 v4 (sources/2609.36279/v4): proves every statement in K3–K8 and K12 under the same names, but with nominal or actuality witnesses that its detector flags (audit/report.txt). It states that the hybrid-free versions are open for the Ax1GenOne variant 3 and for dropping () (§sec:one, §sec:open). Example ex:separating and Separation.lean give a flip-invariant separating model for the empty theory, which is the same technique K11 uses, applied to a different theory.
- C. Benzmüller, "Gödel's and Scott's Variants ... in Lean 4 and TPTP THF", arXiv 2609.26806: the Lean port of the Notes. It does not cover the Ax1Gen readings in variant 3 or detector certificates.
- alozanoroble/godel-ontological-lean and drewarrowood/godel-ontological (GitHub): Lean formalisations of the variants. Neither addresses nominal-free proofs.
- Benzmüller, Proofs Without Nominals, arXiv 2609.36279 v4 (https://arxiv.org/abs/2609.36279):.
- Benzmüller & Scott, Notes on Gödel's and Scott's variants, Monatshefte 2025 (https://link.springer.com/article/10.1007/s00605-025-02078-x): T1, positive properties are possibly exemplified (special case of K13's first half).
- R. Koons, Sobel on Gödel's Ontological Proof (https://robkoons.net/uploads/1/3/5/2/135276253/art_ko the standard argument that a positive property is possibly instantiated.
- alozanoroble/godel-ontological-lean (https://github.com/alozanoroble/godel-ontological-lean): Lean 4 frame-condition table for Gödel, Scott, Anderson, Fitting and Hájek. It does not cover variant 3's essence lemmas.

Our prior-art search (2026-10-22) covered:

- https://arxiv.org/abs/2609.36279: version history (v1–v4, v4 is the latest, 2026-10-14). Fetched 2026-10-22.
- These were search-engine results only. External pages beyond the summaries were not opened.
- I used only search-result summaries for the web hits and did not open the pages.

The search was run automatically by claude/claude-opus-5-5.

The checkers' full answers, with every query and file they searched, are published with this finding (`prior-art/`).

If you know of related work we missed, please open an issue.

We believe this result is new. Our prior-art search (2026-10-22) covered:

- I worked only from search-result summaries for the web results. I did not open those pages.

The search was run automatically by claude/claude-opus-5-5.

The checkers' full answers, with every query and file they searched, are published with this finding (`prior-art/`).

We found no earlier statement of it, but we can't be certain. If you know of prior work, please open an issue.

How it was produced

This result was found by AI research agents (the autoresearch harness). It was proved in Lean; the proof and check files are included so anyone can re-run them.

2 Claims

Claim K1

`In v4's Ax1GenTwoVariant2 (logic K), Ax1Gen_of_Two, Two_equivalent, P_of_conj, loc_of_conj, ae_Two_nominal, G_ex_Two_unconditional_nominal and only_top_and_empty_domain(_False) have proofs of the identical statements that v4's detector passes in witness and whole-term mode.`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K1-hybrid_TwoV2.py`: The embedded Lean file compiles against /library with no sorry, and `python /library/tools/hybrid_check.py -path <tmp> Programme.F.TwoV2 <8 names>` exits 0: no theorem hybrid, inheriting or whole-term flagged. — pass

- `checks/K1-hybrid_TwoV2.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `Two_equivalent` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Evidence.

- proof `checks/K1-TwoV2.lean`: Programme.F.TwoV2.Ax1Gen_of_Two_hf — pass (`results/K1-1.json`)
- proof `checks/K1-TwoV2.lean`: Programme.F.TwoV2.Two_equivalent_hf — pass (`results/K1-2.json`)
- proof `checks/K1-TwoV2.lean`: Programme.F.TwoV2.loc_of_conj_hf — pass (`results/K1-3.json`)
- proof `checks/K1-TwoV2.lean`: Programme.F.TwoV2.P_of_conj_hf — pass (`results/K1-4.json`)
- proof `checks/K1-TwoV2.lean`: Programme.F.TwoV2.ae_Two_nominal_hf — pass (`results/K1-5.json`)
- proof `checks/K1-TwoV2.lean`: Programme.F.TwoV2.G_ex_Two_unconditional_nominal_hf — pass (`results/K1-6.json`)
- proof `checks/K1-TwoV2.lean`: Programme.F.TwoV2.only_top_and_empty_domain_False_hf — pass (`results/K1-7.json`)
- proof `checks/K1-TwoV2.lean`: Programme.F.TwoV2.only_top_and_empty_domain_hf — pass (`results/K1-8.json`)

Prior art. Partial prior art.

Claim K2

`In v4's Ax1GenTwoVariant3Plain (logic K), Ax1Gen_of_Two, Two_equivalent, P_of_conj, loc_of_conj, ae_Two_nominal, G_ex_Two_unconditional_nominal and only_top_and_empty_domain(_False) have proofs of the identical statements that v4's detector passes in witness and whole-term mode.`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K2-hybrid_TwoV3Plain.py`: The embedded Lean file compiles against /library with no sorry, and `python /library/tools/hybrid_check.py -path <tmp> Programme.F.TwoV3Plain <8 names>` exits 0: no theorem hybrid, inheriting or whole-term flagged. — pass
- `checks/K2-hybrid_TwoV3Plain.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by

FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `Two_equivalent` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Evidence.

- proof `checks/K2-TwoV3Plain.lean`: Programme.F.TwoV3Plain.Ax1Gen_of_Two_hf — pass (`results/K2-1.json`)
- proof `checks/K2-TwoV3Plain.lean`: Programme.F.TwoV3Plain.Two_equivalent_hf — pass (`results/K2-2.json`)
- proof `checks/K2-TwoV3Plain.lean`: Programme.F.TwoV3Plain.loc_of_conj_hf — pass (`results/K2-3.json`)
- proof `checks/K2-TwoV3Plain.lean`: Programme.F.TwoV3Plain.P_of_conj_hf — pass (`results/K2-4.json`)
- proof `checks/K2-TwoV3Plain.lean`: Programme.F.TwoV3Plain.ae_Two_nominal_hf — pass (`results/K2-5.json`)
- proof `checks/K2-TwoV3Plain.lean`: Programme.F.TwoV3Plain.G_ex_Two_unconditional_nomin — pass (`results/K2-6.json`)
- proof `checks/K2-TwoV3Plain.lean`: Programme.F.TwoV3Plain.only_top_and_empty_domain_ — pass (`results/K2-7.json`)
- proof `checks/K2-TwoV3Plain.lean`: Programme.F.TwoV3Plain.only_top_and_empty_domain_ — pass (`results/K2-8.json`)

Prior art. Partial prior art.

Claim K3

`In v4's Ax1GenBoxVariant3, MC_S4, Th5_S4 and Ax1Gen_of_refl have proofs of the identical statements that v4's detector passes (witness and whole-term, full names); the reflexivity hypothesis is not used.`

Status: proved (axioms: Ax1GenBox, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K3-hybrid_BoxV3.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term

mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_S4` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Evidence.

- proof `checks/K3-BoxV3.lean`: Programme.F.BoxV3.MC_S4_hf — pass (`results/K3-1.json`)
- proof `checks/K3-BoxV3.lean`: Programme.F.BoxV3.Ax1Gen_of_refl_hf — pass (`results/K3-2.json`)
- proof `checks/K3-BoxV3.lean`: Programme.F.BoxV3.Th5_S4_hf — pass (`results/K3-3.json`)

Prior art. Partial prior art.

Claim K4

`In v4's GoedelVariantHOML3inS4oneFile, AllExist, Th3, Th5, MC, MCdual, Triv, UniqueEss1/2/3, EssMem, EssMemI, EssSupp, EssDetermined and Ess_iff_Ess2 have proofs of the identical statements that v4's detector passes (witness and whole-term, full names), using neither Rrefl nor Rtrans.`

Status: proved (axioms: Ax1Gen, Ax2a, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K4-hybrid_HOML3.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `Th3` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)
- `checks/K4-hybrid_HOML3.py`: Compiles the embedded sha-pinned file against /library (no error/sorry); v4's HybridAudit (witness + whole-term) flags none of the 14 theorems by full name (0 hybrid/inheriting/EXTRA); the transitive constant closure of every proof avoids all F_WORKLIST names of the module incl. ACTUAL-only AllExist and AllExistI; negative control (re-export of Th3) is flagged. — pass

Evidence.

- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.AllExist_hf — pass (`results/K4-16.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.Th3_hf — pass (`results/K4-17.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.Th5_hf — pass (`results/K4-18.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.MC_hf — pass (`results/K4-19.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.MCdual_hf — pass (`results/K4-20.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.Triv_hf — pass (`results/K4-21.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.UniqueEss1_hf — pass (`results/K4-22.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.UniqueEss2_hf — pass (`results/K4-23.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.UniqueEss3_hf — pass (`results/K4-24.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.EssMemI_hf — pass (`results/K4-25.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.EssMem_hf — pass (`results/K4-26.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.Ess_iff_Ess2_hf — pass (`results/K4-27.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.EssSupp_hf — pass (`results/K4-28.json`)
- proof `checks/K4-HOML3.lean`: Programme.F.HOML3.EssDetermined_hf — pass (`results/K4-29.json`)

Prior art. Partial prior art.

Claim K5

`In v4's Ax1GenOneVariant3 (logic K), MC_unconditional, MC_One, AllActual_of_PBot, NotPBot/PTop/G_ex/Th4_One_deriv, E_top_of_PBot, AllExist_One, One_gives_S, One_gives_Box, Box_gives_One have proofs of the identical statements that v4's detector passes (full names).`

Status: proved (axioms: Ax1GenOne, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K5-hybrid_OneV3.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_unconditional` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Evidence.

- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.MC_unconditional_hf — pass (`results/K5-1.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.MC_One_hf — pass (`results/K5-2.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.AllActual_of_PBot_hf — pass (`results/K5-3.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.NotPBot_One_deriv_hf — pass (`results/K5-4.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.PTop_One_deriv_hf — pass (`results/K5-5.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.E_top_of_PBot_hf — pass (`results/K5-6.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.G_ex_One_deriv_hf — pass (`results/K5-7.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.Th4_One_deriv_hf — pass (`results/K5-8.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.AllExist_One_hf — pass (`results/K5-9.json`)

- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.One_gives_S_hf — pass (`results/K5-10.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.One_gives_Box_hf — pass (`results/K5-11.json`)
- proof `checks/K5-OneV3.lean`: Programme.F.OneV3.Box_gives_One_hf — pass (`results/K5-12.json`)

Prior art. Partial prior art.

Claim K6

`In v4's Ax1GenTwoVariant3PlainPoss and Ax1GenTwoVariant3PlainAndersonQuant, MC_unconditional has a proof of the identical statement that v4's detector passes (witness and whole-term, full names).`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K6-hybrid_TwoV3PlainPoss.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_unconditional` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)
- `checks/K6-hybrid_TwoV3PlainAndersonQuant.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_unconditional` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Evidence.

- proof `checks/K6-TwoV3PlainPoss.lean`: Programme.F.TwoV3PlainPoss.MC_unconditional_hf — pass (`results/K6-1.json`)
- proof `checks/K6-TwoV3PlainAndersonQuant.lean`: Programme.F.TwoV3PlainAndersonQuant.MC_u — pass (`results/K6-3.json`)

Prior art. Partial prior art.

Claim K7

In v4's Ax1GenTwoVariant3 (logic K), MC_unconditional, MC_Two, L_Two_hybrid, G_ex_Two_actual_hybrid and Th4_Two_actual_hybrid have proofs of the identical statements that v4's detector passes (witness and whole-term, full names), via a hybrid-free lemma that every σ is constant.

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K7-hybrid_TwoV3.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `MC_unconditional` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Evidence.

- proof `checks/K7-TwoV3.lean`: Programme.F.TwoV3.MC_unconditional_hf — pass (`results/K7-1.json`)
- proof `checks/K7-TwoV3.lean`: Programme.F.TwoV3.MC_Two_hf — pass (`results/K7-2.json`)
- proof `checks/K7-TwoV3.lean`: Programme.F.TwoV3.L_Two_hybrid_hf — pass (`results/K7-3.json`)
- proof `checks/K7-TwoV3.lean`: Programme.F.TwoV3.G_ex_Two_actual_hybrid_hf — pass (`results/K7-4.json`)
- proof `checks/K7-TwoV3.lean`: Programme.F.TwoV3.Th4_Two_actual_hybrid_hf — pass (`results/K7-5.json`)

Prior art. Partial prior art.

Claim K8

`In v4's Ax1GenTwoVariant3Th4, G_ex_Two_hybrid and AllExist_Two have proofs of the identical statements that v4's detector passes (witness and whole-term, full names), with no frame condition.`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax2b, Ax3, Ax4, P, R, Th4, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K8-hybrid_TwoV3Th4.py`: Strict full-name certification (the skeptic): compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that no named theorem appears, by FULLY QUALIFIED name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's hybrid `G_ex_Two_hybrid` in the same namespace IS flagged, so the matcher is live. — pass (Lean-proved, no sorryAx)

Evidence.

- proof `checks/K8-TwoV3Th4.lean`: Programme.F.TwoV3Th4.G_ex_Two_hybrid_hf — pass (`results/K8-1.json`)
- proof `checks/K8-TwoV3Th4.lean`: Programme.F.TwoV3Th4.AllExist_Two_hf — pass (`results/K8-2.json`)

Prior art. Partial prior art.

Claim K11

On worlds Bool, one always-actual entity, universal R, $P\varphi w := \varphi()w$, swap-invariant domains closed under all terms of an $\mathcal{L}$-datatype: Ax1, Ax2a, Ax2b, Ax3, both Ax4, Ax1Gen(Two/One/Box), Th4 hold, E holds at both worlds, so NoNecExist fails.

Status: proved (axioms: propext, Classical.choice, Quot.sound).

Verified. No check scripts.

Evidence.

- proof `checks/K11-NoNecModel.lean`: Programme.F.NoNecModel.nonec_model — pass (`results/K11-1.json`)

Prior art. Partial prior art.

Claim K12

`In v4's Ax1GenTwoVariant3Plain (logic K), AllExist and MC_OneWorld have proofs of the identical statements that v4's detector passes (witness and whole-term, full names).`

Status: proved (axioms: Ax1GenTwo, Ax2a, Ax3, Ax4, P, R, e, eNonempty, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K12-hybrid_V3PlainExtra.py`: Strict full-name certification: compiles the embedded, sha-pinned Lean file against /library with no error/sorry; runs v4's HybridAudit (#hybrid_audit and #hybrid_all) on the module and asserts that neither theorem appears, by fully qualified name, as hybrid, inheriting or EXTRA, that whole-term mode reports and that there is no sorryAx; negative control: a re-export of the library's inheriting `Two_equivalent` in the same namespace IS flagged. — pass (Lean-proved, no sorryAx)

Evidence.

- proof `checks/K12-V3PlainExtra.lean`: Programme.F.V3PlainExtra.AllExist_hf — pass (`results/K12-1.json`)
- proof `checks/K12-V3PlainExtra.lean`: Programme.F.V3PlainExtra.MC_OneWorld_hf — pass (`results/K12-2.json`)

Prior art. Partial prior art.

Claim K13

Abstract Henkin variant 2, exclusive Ax2a, Ax2b, Ax4, Ax1GenR over admitted rigid families $\mathcal{F}$, any actuality: each $\Phi \in \mathcal{F}$ positive at w is realised by an actual entity at an R-successor; if G is admitted and $\neg PG\,w$, no $\Phi \in \mathcal{F}$ equals admitted P_w.

Status: proved (axioms: propext, Classical.choice, Quot.sound).

Verified. No check scripts.

Evidence.

- proof `checks/K13-RigidLemma.lean`: Programme.F.RigidLemma.realised — pass (`results/K13-1.json`)
- proof `checks/K13-RigidLemma.lean`: Programme.F.RigidLemma.no_god_succ — pass (`results/K13-2.json`)
- proof `checks/K13-RigidLemma.lean`: Programme.F.RigidLemma.pos_type_omitted — pass (`results/K13-3.json`)
- proof `checks/K13-RigidLemma.lean`: Programme.F.RigidLemma.pos_type_omitted_of_pred — pass (`results/K13-4.json`)

Prior art. Partial prior art.

Claim K14

In v4's Ax1GenRigidVariant2 (logic K), L_R (Ax1GenR → $\lfloor PG \rfloor$) has a proof of the identical statement that v4's detector passes (witness and whole-term, full names), with witness PosGlobal, from Ax2a, Ax2b, Ax4 only, with no condition on positivity.

Status: proved (axioms: Ax2a, Ax2b, Ax4, P, R, e, existsAt, i, propext, Classical.choice, Quot.sound).

Verified.

- `checks/K14-hybrid_RigidHF.py`: Compiles the embedded sha-pinned file against /library (no error/sorry); v4's HybridAudit (witness + whole-term) flags neither theorem by full name (0 hybrid/inheriting/EXTRA); proof closure avoids L_R, RigidPos, RigidPos_of_link, pos_link, Link, L_R_rigidpos; negative control (re-export of L_R) is flagged. — pass

Evidence.

- proof `checks/K14-RigidHF.lean`: Programme.F.RigidHF.L_R_hf — pass (`results/K14-1.json`)
- proof `checks/K14-RigidHF.lean`: Programme.F.RigidHF.guard_pos_hf — pass (`results/K14-2.json`)

Prior art. Possibly new: no prior art found in the owner's search.

Provenance. Run `20261022-1314`, claims K1, K2, K3, K4, K5, K6, K7, K8, K11, K12, K13, K14, run-folder commit `7e239585d6c77726d88783754d4bfa7d3c3bf197`, published 2026-10-24.

3 How to verify

Each finding has its check scripts in `claims/ontological-nominal-free-proofs/checks/` of the findings repository. Run each with `python -I <script>`; it passes if it exits with status 0. The checks are Python scripts and need the packages of the research image (the image's `requirements.txt`); the Lean checks also need `lean`.

4 Revisions

- 2026-10-24 — readable prior-art section

References

[1] Mihaly Barasz, Paul Christiano, Benja Fallenstein, Marcello Herreshoff, Patrick LaVictoire, and Eliezer Yudkowsky. Robust cooperation in the prisoner's dilemma: Program equilibrium via provability logic, 2014.

[2] Paul Beame and Søren Riis. More on the relative strength of counting principles. In *Proof Complexity and Feasible Arithmetics*, volume 39 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 13–35. American Mathematical Society, 1998.

[3] Christoph Benzmüller. A (simplified) supreme being necessarily exists, says the computer: Computationally explored variants of gödel's ontological argument. In *Proceedings of KR 2020*, 2020.

[4] Christoph Benzmüller. A comment on modal collapse and ultrafilters in gödel's ontological argument. arXiv:2608.07578, 2026.

[5] Christoph Benzmüller. Gödel's and scott's variants of the ontological argument in lean 4 and tptp thf. arXiv:2609.26806, 2026.

[6] Christoph Benzmüller. Proofs without nominals: Gödel's ontological argument, its shallow embedding, and the open questions of the monatshefte notes. arXiv:2609.36279v4, 2026.

[7] Christoph Benzmüller and David Fuenmayor. Computer-supported analysis of positive properties, ultrafilters and modal collapse in variants of gödel's ontological argument. arXiv:1910.08955, 2019.

[8] Christoph Benzmüller and Lawrence C. Paulson. Quantified multimodal logics in simple type theory. *Logica Universalis*, 7(1), 2013.

[9] Christoph Benzmüller and Dana S. Scott. Notes on gödel's and scott's variants of the ontological argument. *Monatshefte für Mathematik*, 2025.

[10] Christoph Benzmüller, Leon Weber, and Bruno Woltzenlogel Paleo. Computer-assisted analysis of the anderson–hájek ontological controversy. *Logica Universalis*, 2017.

[11] Patrick Blackburn, Maarten de Rijke, and Yde Venema. *Modal Logic*. Cambridge University Press, 2001.

[12] George Boolos. *The Logic of Provability*. Cambridge University Press, 1993.

[13] Andrew Critch. A parametric, resource-bounded generalization of löb's theorem, and a robust cooperation criterion for open-source game theory. *Journal of Symbolic Logic*, 2019.

[14] Andrew Critch, Michael Dennis, and Stuart Russell. Cooperative and uncooperative institution designs: Surprises and problems in open-source game theory. arXiv:2208.07006, 2022.

[15] Kit Fine. Propositional quantifiers in modal logic. *Theoria*, 36(3), 1970.

[16] Melvin Fitting. *Types, Tableaus, and Gödel's God*. Kluwer, 2002.

[17] Murdoch J. Gabbay and Dominic P. Mulligan. Nominal Henkin semantics: simply-typed lambda-calculus models in nominal sets. arXiv:1111.0089, 2011.

[18] Gram_Stone. Modal chicken. LessWrong, 2015.

[19] Donald Hobson. Yet more modal combat. LessWrong / AI Alignment Forum, 2021.

[20] Wesley H. Holliday. A note on algebraic semantics for S5 with propositional quantifiers. *Notre Dame Journal of Formal Logic*, 60(2), 2019.

[21] Patrick LaVictoire, Benja Fallenstein, Eliezer Yudkowsky, Mihaly Barasz, Paul Christiano, and Marcello Herreshoff. Program equilibrium in the prisoner's dilemma via löb's theorem. In *AAAI Workshop on Multiagent Interaction without Prior Coordination*, 2014.

[22] Alex Mennen. Modal combat for games other than the prisoner's dilemma. LessWrong / AI Alignment Forum, 2017.

[23] Mladen Mika and Jakob Nordström. A generalized method for proving polynomial calculus degree lower bounds. In *30th Conference on Computational Complexity (CCC 2015)*, volume 33 of *LIPIcs*, pages 467–487, 2015.

[24] Reinhard Muskens. Intensional models for the theory of types. *Journal of Symbolic Logic*, 72(1), 2007.

[25] Caspar Oesterheld. A note on the compatibility of different robust program equilibria of the prisoner's dilemma, 2022.

[26] Alexander A. Razborov. Proof complexity of pigeonhole principles. In *Developments in Language Theory (DLT 2001)*, volume 2295 of *Lecture Notes in Computer Science*, pages 100–116. Springer, 2002.

[27] Søren Riis. *Independence in Bounded Arithmetic*. PhD thesis, University of Oxford, 1993.

[28] Søren Riis and Meera Sitharam. Non-constant degree lower bounds imply linear degree lower bounds. Technical Report TR97-048, Electronic Colloquium on Computational Complexity, 1997.

[29] Robert M. Solovay. Provability interpretations of modal logic. *Israel Journal of Mathematics*, 25:287–304, 1976.

[30] Aleksandar Todorov, Jesse ten Napel, and Alexander Müller. Parametric open source games. arXiv:2606.27068, 2026.

[31] transhumanist_atom_understander. Is there only one FairBot? LessWrong, with a comment by Zac Hatfield-Dodds, 2026.